



Web Programming For Back End - Unidade III

Olá, estudantes!

Sejam bem-vindos(as) à **unidade III** da disciplina **Web Programming For Back End**!

A disciplina **Web Programming For Back End** oferece uma visão aprofundada sobre o desenvolvimento de aplicações robustas e escaláveis. Exploraremos conceitos essenciais, como **APIs e seu papel no Back-End**, permitindo a comunicação eficiente entre sistemas. Também abordaremos o **padrão de projeto MVC (Model-View-Controller)**, fundamental para a organização e manutenção do código. Além disso, estudaremos bibliotecas essenciais, como **cors**, para gerenciamento de permissões entre origens, e **body-parser**, que facilita o tratamento de requisições. Essa jornada permitirá aos alunos desenvolverem aplicações modernas e preparadas para os desafios do mundo real.

Vamos nessa!



Aula 1: Introdução a API



Aula 2: Planejar uma Arquitetura de Projetos Aplicada ao Back-End



Download do Conteúdo

Aula 1: Introdução a API

Videoaula 17 - Entendendo o conceito de API

Nesta videoaula, você entenderá o que são **APIs** e por que elas são fundamentais no desenvolvimento de software moderno. Exploraremos como elas permitem a comunicação entre sistemas, facilitando a integração e o compartilhamento de dados. Com exemplos práticos, você verá como as APIs tornam aplicações mais dinâmicas, escaláveis e eficientes no mundo digital.

Aperte o play e confira!



QUAL É A IMPORTÂNCIA PARA SUA ATUAÇÃO PROFISSIONAL? —

Saber conhecer e aplicar o conceito de API é essencial para o desenvolvimento Back-End, o aluno precisa ter ciência das características principais de como planejar e criar uma API de qualidade.

Conceito de API e seu Papel no Back-End

API (Application Programming Interface) é um conjunto de regras que permite que diferentes sistemas ou partes de um sistema se comuniquem entre si. No contexto de back-end, a API atua como uma ponte entre o servidor (onde os dados e regras de negócios residem) e o cliente (como front-end, dispositivos móveis, ou outros sistemas).

1

O que é uma API no Back-End?

No back-end, a API define os endpoints e métodos que o cliente pode usar para interagir com os dados ou executar ações.

Essas interações normalmente seguem os padrões HTTP, com métodos como:

- GET: Recuperar dados (ex.: Buscar lista de produtos)

- POST: Criar novos recursos (ex.: Adicionar um produto)
- PUT/PATCH: Atualizar recursos existentes (ex.: Alterar detalhes de um produto)
- DELETE: Excluir recursos (ex.: Remover um produto)

Esses métodos são encapsulados em endpoints (URLs específicas), como: /api/products.

2

Tipos de APIs no Contexto de Back-End

REST (Representational State Transfer) —

- Baseado em recursos e usa URLs simples.
- Os dados são geralmente retornados em JSON.
- É o mais comum devido à sua simplicidade e eficiência.

GraphQL —

- Permite consultas específicas e personalizadas para os dados que você deseja.
- Menos solicitações redundantes e mais controle para o cliente sobre o que receber.

SOAP (Simple Object Access Protocol) —

- Um padrão mais antigo, baseado em XML.
- Fornece mais estrutura e segurança robusta, usado frequentemente em sistemas corporativos.

WebSockets —

- Comunicação bidirecional e em tempo real entre cliente e servidor.
- Muito usado em aplicações de chat e notificações.

Componentes Principais de APIs Back-End



Endpoints: URLs que representam recursos (ex.: /api/users).



Controladores: Gerenciam a lógica dos endpoints, processando as requisições.



Modelos: Estruturas de dados usadas para representar os recursos no sistema.



Serviços: Camada de lógica de negócios que processa os dados e regras antes de retorná-los ao cliente.



Autenticação/Autorização: Proteção de APIs por meio de tokens, JWT, OAuth, ou API keys.



Validação de Dados: Garantir que os dados enviados pelo cliente sejam corretos antes de serem processados.



Logs e Monitoramento: Para rastrear chamadas de API e analisar desempenho ou erros.

4

Fluxo de uma Chamada de API Back-End



O cliente faz uma requisição HTTP para um endpoint (ex.: GET /api/products).



O servidor recebe e processa a solicitação.



O controlador chama a camada de serviço, que aplica a lógica de negócios.



A API interage com a base de dados (se necessário).



A resposta é retornada para o cliente em JSON (ou outro formato).

5

Práticas Comuns para

Desenvolvimento de APIs Back-End

 **Versionamento:** Use versões em URLs (ex.: /api/v1/products) para manter compatibilidade com diferentes clientes.

 **Documentação:** Utilize ferramentas como Swagger ou Postman para documentar e testar endpoints.

 **Segurança:** Implementar HTTPS, CORS, autenticação robusta e proteção contra injeção de SQL.

 **Paginação:** Para grandes conjuntos de dados, implemente paginação (ex.: ?page=2&limit=20).

 **Taxas de Limite (Rate Limiting):** Para evitar abuso, limite o número de solicitações por cliente em um período de tempo.

 **Erros Uniformes:** Padronize os erros para facilitar o tratamento por clientes (ex.: 404 Not Found, 500 Internal Server Error).

Exemplo Simples de uma API RESTful

Imagine que você tem um sistema de gerenciamento de livros:

Endpoint: GET /api/books

Resposta (JSON):

```
[  
  
  {  
  
    "id": 1,  
  
    "title": "Clean Code",  
  
    "author": "Robert C. Martin"  
  
  },  
  
  {  
  
    "id": 2,
```

```
"title": "The Pragmatic Programmer",  
  
"author": "Andrew Hunt"  
  
}  
  
]
```

Endpoint: POST /api/books

Payload (JSON):

```
{  
  
"title": "Design Patterns",  
  
"author": "Erich Gamma"  
  
}
```

Resposta:

201 Created

```
{  
  
  "id": 3,  
  
  "title": "Design Patterns",  
  
  "author": "Erich Gamma"  
  
}
```

7

Regras e Práticas para a Criação de Endpoints no Padrão REST

O padrão **REST (Representational State Transfer)** tem um conjunto de boas práticas para a criação de endpoints. Essas práticas ajudam a criar APIs mais previsíveis, fáceis de manter e com boa experiência para os desenvolvedores que as consomem.

Nomeação dos Endpoints

- Use Substantivos no Plural para Recursos;

- Evite usar verbos ou nomes no singular para os endpoints. O endpoint deve representar o recurso.

CORRETO:	ERRADO:
GET /api/users → Lista todos os usuários POST /api/users → Cria um usuário	
CORRETO:	ERRADO:
GET /api/getUsers POST /api/createUser	

Hierarquia e Relacionamentos

Use uma hierarquia clara para representar os relacionamentos entre recursos.

CORRETO:	ERRADO:

GET /api/users/1/orders → Retorna os pedidos do usuário com ID 1

CORRETO:

ERRADO:

GET /api/getOrdersByUserId/1

Utilize IDs para Identificação de Recursos

Quando for necessário acessar um recurso específico, use um identificador único no endpoint.

Exemplo: GET /api/users/123 → Retorna o usuário com ID 123

Filtros, Ordenação e Paginação

Ao lidar com grandes quantidades de dados, forneça mecanismos para filtrar, ordenar e paginar os resultados.

FILTROS	ORDENAÇÃO:	PAGINAÇÃO:
---------	------------	------------

Use parâmetros de consulta (query params).

GET /api/users?age=25 → Retorna usuários com 25 anos

FILTROS	ORDENAÇÃO:	PAGINAÇÃO:
---------	------------	------------

GET /api/users?sort=name → Ordena por nome

GET /api/users?sort=-name → Ordena por nome em ordem decrescente

FILTROS	ORDENAÇÃO:	PAGINAÇÃO:
---------	------------	------------

GET /api/users?page=2&limit=10 → Retorna a página 2 com 10 itens por página

Versionamento

Sempre versione sua API para evitar que mudanças quebrem funcionalidades existentes. Use a versão na URL.

- GET /api/v1/users
- GET /api/v2/users

Tratamento de Erros

Use códigos de status HTTP padronizados para indicar o resultado das requisições:

- **200 OK:** Requisição bem-sucedida.
- **201 Created:** Recurso criado com sucesso.
- **204 No Content:** Recurso excluído com sucesso.
- **400 Bad Request:** Requisição inválida ou erro de validação.
- **401 Unauthorized:** O cliente não está autenticado.
- **403 Forbidden:** O cliente não tem permissão para acessar o recurso.

- **404 Not Found:** Recurso não encontrado.
- **500 Internal Server Error:** Erro inesperado no servidor.

Videoaula 18 - Preparando ambiente para a criação da API

Antes de começar a desenvolver sua própria API, é essencial configurar um ambiente adequado. Nesta videoaula, você aprenderá a instalar as ferramentas necessárias, configurar o servidor e organizar seu projeto para garantir um fluxo de trabalho eficiente. Esse primeiro passo é fundamental para um desenvolvimento ágil e sem complicações.



QUAL É A IMPORTÂNCIA PARA SUA ATUAÇÃO PROFISSIONAL? —

Criar uma API é fundamental para a integração entre o back-end e front-end.

Bibliotecas para criar a API em node.js

Express

A biblioteca mais utilizada no Node.js é o Express.js. O Express é amplamente adotado por desenvolvedores devido à sua simplicidade, flexibilidade e grande comunidade, o que facilita encontrar tutoriais, exemplos e soluções para problemas comuns.

Por que o Express é tão popular?

- **Simplicidade:** Fácil de aprender e configurar.
- **Flexibilidade:** Pode ser usado para APIs REST, aplicações web, ou até servidores mais complexos.
- **Ecosistema:** Existem muitos middlewares prontos que ampliam suas funcionalidades.
- **Comunidade:** Por ser maduro e muito utilizado, há grande suporte de fóruns e documentação.

Fastify

O Fastify é um framework web para Node.js focado em desempenho e eficiência. Ele foi inspirado em frameworks como Hapi e Express, mas com ênfase em ser mais rápido e seguro. Benchmarks indicam que o Fastify pode lidar com um número significativamente maior de requisições por segundo em comparação com o Express.

Principais características do Fastify:

- **Alto desempenho:** Projetado para ser rápido, lidando com mais requisições por segundo.

- **Extensibilidade:** Possui um sistema de plugins robusto que facilita a adição de funcionalidades.
- **Validação de esquemas:** Utiliza JSON Schema para validação de dados, garantindo maior segurança e consistência.
- **Suporte a TypeScript:** Compatível com TypeScript, oferecendo uma melhor experiência de desenvolvimento para quem utiliza essa linguagem.

O Fastify é utilizado por empresas como Capital One, Walmart e American Express, demonstrando sua confiabilidade em ambientes de produção.

Instalação do Express e Criação de uma API exemplo em Node.js

Pré-requisitos

Certifique-se de ter o seguinte instalado:

- Node.js (Versão 12 ou superior)
- NPM (Node Package Manager) – geralmente vem com o Node.js

Instalação

```
<npm install express --save
```

Processo básico para a criação de um servidor utilizando o Express

```
// Importa o Express

const express = require('express');

// Cria a aplicação Express

const app = express();

// Define uma porta para o servidor

const PORT = 8080;
```

```
// Cria uma rota de teste

app.get('/', (req, res) => {

  res.send('API funcionando!');

});

// Inicia o servidor

app.listen(PORT, () => {

  console.log(`Servidor rodando em  
http://localhost:${PORT}`);

});
```

No terminal, execute o comando para iniciar o servidor:

```
node server.js
```

Você verá a mensagem no terminal indicando que o servidor está rodando:

Servidor rodando em <http://localhost:8080>

Abra o navegador ou uma ferramenta como o Postman e acesse <http://localhost:8080>. Você verá a mensagem API funcionando!.

Leitura direto da fonte

APIs REST

Seus serviços prontos para
o mundo real



 Casa do
Código

ALEXANDRE SAUDATE

Capa do livro

APIs REST: seus serviços prontos para o mundo real

APIs REST: Seus Serviços Prontos para o Mundo Real é um guia essencial para desenvolvedores que desejam criar APIs robustas, seguras e escaláveis. O livro aborda desde os conceitos fundamentais do REST até práticas avançadas, garantindo que os serviços desenvolvidos estejam preparados para demandas reais do mercado. Com exemplos práticos e uma abordagem clara, a obra explora autenticação, versionamento, segurança e boas práticas para construção de APIs eficientes. Ideal para quem busca aprimorar suas habilidades e entregar soluções modernas e bem estruturadas, o livro é um recurso indispensável para profissionais de tecnologia.

Boa Leitura 📖📖

Acesso ao Livro na Biblioteca Virtual UniFECAF

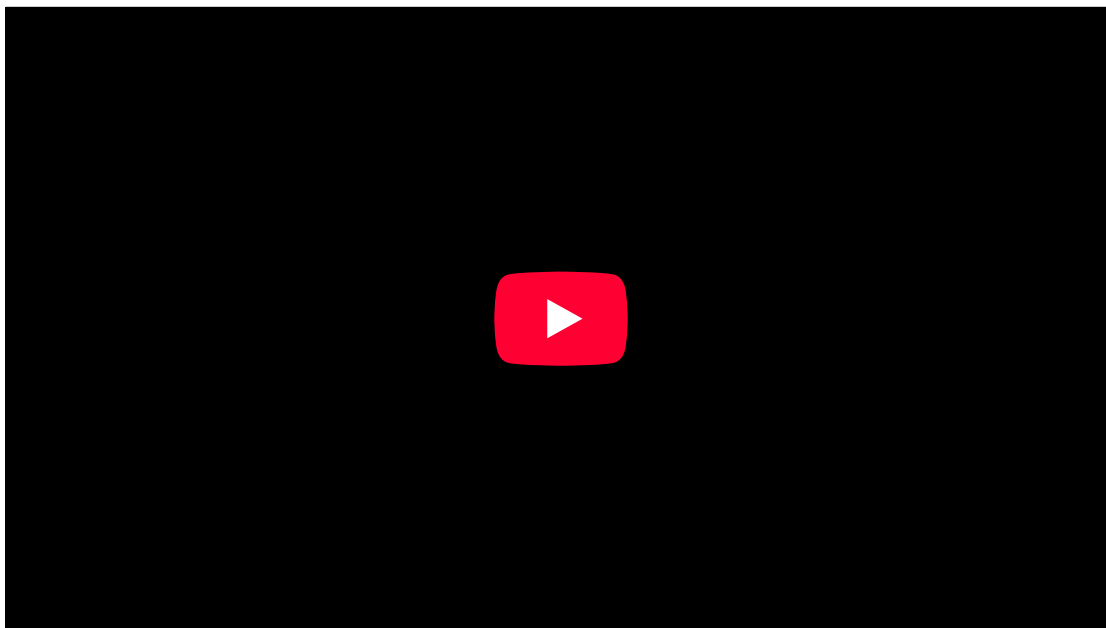
Para acessar o livro e realizar a leitura clique no botão ao lado.

ACESSAR O LIVRO

Saiba mais

Neste vídeo você irá compreender melhor como funciona o padrão REST para a concepção de uma API, além de entender como uma API retorna o seu status code nas requisições.

- O que é o padrão REST
- Como o padrão REST interage com o protocolo HTTP
- Padrões de status code para retorno de APIs REST



Prepare-se para mais uma jornada de conhecimento 🧐

Nos vemos na próxima aula!

CLIQUE AQUI

Aula 2: Planejar uma Arquitetura de Projetos Aplicada ao Back-End

Videoaula 19 - Conhecendo o modelo MVC para arquiteturas de projetos

O padrão MVC (Model-View-Controller) é uma das arquiteturas mais utilizadas no desenvolvimento de software. Nesta videoaula, você conhecerá como esse modelo organiza a estrutura do código, separando a lógica da aplicação, a interface e o controle das requisições. Com essa abordagem, sua API se tornará mais modular, manutenível e escalável.

Aperte o play e confira!



QUAIS HABILIDADES SERÃO DESENVOLVIDAS A PARTIR DESSES TÓPICOS? —

- Planejar uma arquitetura de projetos aplicada ao back-end
- Construção do projeto aplicando o modelo MVC

O padrão de projeto MVC

O modelo MVC (Model-View-Controller) pode ser aplicado ao desenvolvimento de back-end usando Node.js para criar uma arquitetura organizada e escalável.

Estrutura do MVC no Back-end com Node.js

No contexto de uma aplicação back-end em Node.js, cada parte do MVC funciona da seguinte maneira:

Model (Modelo)

Gerencia os dados e a lógica de negócios. No Node.js, geralmente utilizamos bibliotecas como Mongoose (para MongoDB) ou Sequelize ou prisma (para SQL) para criar modelos e definir esquemas de dados.

Funções comuns:

- Consultar/atualizar banco de dados.
- Validar dados.

- Lógica de regras de negócio.

Controller (Controlador)

- Recebe as solicitações (requests) do cliente.
- Processa os dados de entrada.
- Invoca as funções apropriadas do Model.
- Devolve uma resposta para o cliente, seja um JSON ou uma página renderizada.
- View (Visão).

No back-end com Node.js, as Views geralmente são páginas renderizadas dinamicamente (usando EJS, Pug ou Handlebars) ou respostas JSON para APIs REST. A View não contém lógica de negócios e apenas exibe os dados fornecidos.

O modelo **MVC em Node.js** ajuda a organizar o projeto separando responsabilidades. Isso torna o código mais limpo, fácil de manter e escalável.

Videoaula 20 - Entendendo as bibliotecas de Cors e Body-parser

APIs modernas precisam lidar com permissões e manipulação de dados enviados nas requisições. Nesta videoaula, você entenderá o papel das bibliotecas **CORS**, que controla o acesso de diferentes origens à sua API, e **Body-parser**, responsável por interpretar e processar os dados recebidos. Dominar essas ferramentas é essencial para garantir segurança e eficiência no desenvolvimento de APIs.

Dê o play e confira!



QUAIS HABILIDADES SERÃO DESENVOLVIDAS A PARTIR DESSES TÓPICOS? —

A utilização do cors permite controlar quais origens podem acessar os recursos da sua API., já a biblioteca do body-parser permite analisar os dados recebidos no corpo das requisições HTTP e os converte para formatos manipuláveis no código.

Essas bibliotecas são essenciais para criar APIs que aceitam diferentes formatos de dados e respeitam as políticas de segurança da web.

Biblioteca cors (Cross-Origin Resource Sharing)

O que é CORS?

CORS (Cross-Origin Resource Sharing) é uma política de segurança que controla como os recursos de uma aplicação (servidor) podem ser acessados a partir de domínios diferentes (origens externas).

Por padrão, os navegadores bloqueiam solicitações feitas de um domínio para outro (por exemplo, de <http://meu-front.com> para

<http://minha-api.com>). A biblioteca cors permite que você defina regras para permitir ou restringir essas solicitações.

Como usar cors em uma API Node.js

Para instalar a biblioteca, use o seguinte comando:

```
npm install cors
```

No arquivo principal app.js:

```
const express = require('express');

const cors = require('cors');

const app = express();

// Ativar CORS para todas as origens

app.use(cors());

// Definir uma rota de exemplo

app.get('/api/data', (req, res) => {

  res.json({ message: 'Recurso acessado com CORS habilitado' });
```

```
});  
  
// Iniciar o servidor  
  
const PORT = 3000;  
  
app.listen(PORT, () => {  
  
  console.log(`Servidor rodando em http://localhost:\${PORT}`);  
  
});
```

Com o código acima, a API aceitará solicitações de qualquer domínio.

Exemplo com configuração personalizada

Se quiser permitir apenas solicitações de um domínio específico:

```
app.use(cors({  
  
  origin: 'http://meu-frontend.com', // Permitir apenas este domínio  
  
}));
```

Para habilitar métodos específicos e cabeçalhos personalizados:

```
app.use(cors({  
  
  origin: 'http://meu-frontend.com',  
  
  methods: ['GET', 'POST', 'PUT', 'DELETE'], // Permitir  
  apenas esses métodos  
  
  allowedHeaders: ['Content-Type', 'Authorization'], //  
  Cabeçalhos permitidos  
  
}));
```

O que é o body-parser?

O body-parser é um middleware que analisa (parse) o corpo das requisições HTTP e converte os dados para um formato acessível (normalmente JSON ou URL-encoded). Ele é usado principalmente para lidar com dados enviados em requisições POST ou PUT.

Como usar o body-parser

Instale a biblioteca:

```
npm install body-parser
```

No arquivo principal (app.js):

```
const express = require('express');

const bodyParser = require('body-parser');

const app = express();

// Middleware para analisar o corpo da requisição como JSON

app.use(bodyParser.json());

// Middleware para analisar dados em formato application/x-www-
form-urlencoded

app.use(bodyParser.urlencoded({ extended: true }));

// Rota de exemplo que processa um POST

app.post('/api/tarefas', (req, res) => {

  const { titulo, descricao } = req.body;
```

```
res.json({ message: `Tarefa recebida: ${titulo}, ${descricao}` });

});

// Iniciar o servidor

const PORT = 3000;

app.listen(PORT, () => {

  console.log(`Servidor rodando em http://localhost:\${PORT}`);

});
```

Explicação das Configurações

bodyParser.json()

- Analisa o corpo da requisição com Content-Type: application/json.
- Transforma o corpo em um objeto JavaScript acessível via req.body.

bodyParser.urlencoded({ extended: true })

- Analisa dados enviados no formato URL-encoded (formulários HTML).
- extended: true permite a análise de objetos aninhados.

Alternativa: `express.json()` e `express.urlencoded()`

Desde a versão 4.16 do Express, o `body-parser` foi incorporado ao próprio Express. Ou seja, você pode usar diretamente:

```
app.use(express.json()); // Analisa JSON

app.use(express.urlencoded({ extended: true })); // Analisa
dados de formulários
```

Testando os aprendizados

Que tal testar os aprendizados desta unidade? Vem comigo!

Com base nos estudos realizados nesta unidade, marque VERDADEIRO ou FALSO nas perguntas/afirmativas abaixo:

"No modelo MVC, o Controller é responsável por acessar diretamente o banco de dados para buscar ou manipular os dados."

☐ Verdadeiro

☐ Falso

SUBMIT

"Uma API back-end pode retornar tanto dados em formato JSON quanto HTML renderizado para o cliente, dependendo da implementação."

☐ Verdadeiro

☐ Falso

SUBMIT

Aula Ao Vivo

Quantos aprendizados até aqui, não é mesmo!?

Agora, te convido a participar da aula ao vivo 🎬

Confira abaixo os temas que serão abordados, você não pode perder! 👩👨

🎬	Aplicação dos conceitos de API utilizando o node.js,
🎬	Mão na Massa - Criando uma API
🎬	Mão na Massa - Criando uma estrutura de diretórios pelo modelo MVC

Essas atividades tornam as aulas ao vivo mais dinâmicas e produtivas, estimulando o protagonismo dos estudantes, consolidando o aprendizado e fortalecendo a reflexão crítica sobre a articulação entre teoria e prática.

Sintonia do Saber

Nesse podcast você irá compreender a devida importância da utilização de uma API aplicada a projetos reais de mercado.

Dê o play e ouça 🎧



Leitura Recomendada

Olá, estudantes! 📖🔍

Neste material o aluno poderá conhecer mais sobre o modelo de projeto REST aplicado na construção de APIs.

AWS

O que é uma API RESTful?

[CLIQUE AQUI](#)

Parabéns, você finalizou a unidade 😊

Faça download do conteúdo!

CLIQUE AQUI

Download do Conteúdo

Que tal baixar todo esse conteúdo e aproveitar os estudos
no seu melhor momento?

Faça o download do conteúdo dessa unidade:



BAIXE O CONTEÚDO DESTA UNIDADE.pdf
1.7 MB

